

SmartMess

Product Requirements Document

Consumption-aware mess billing & operations platform

1 Higher-order goal

This project aims to make hostel dining fairer and more resource-efficient by giving students and mess operators a shared, real-time signal of who is actually eating. While the broader goal is campus resource efficiency and financial fairness for students, the immediate engineering objective is to translate *consumption-aware billing and demand planning* into a measurable, deployable software platform.

2 Time-to-value

- We will deliver meaningful increments quickly by shipping consumption tracking and billing transparency as a working pilot in a single hostel mess before building the admin and analytics layers.
- We will prioritize fast feedback over completeness by validating each phase's release criteria (e.g., billing view matches manual reconciliation) before starting the next phase.
- Success is defined by what can be delivered and measured in the first phase — reliable opt-in/opt-out and accurate billing — then improved based on pilot feedback.

3 Problem Statement

Students in hostels are charged a fixed mess fee regardless of how many meals they actually eat, while mess operators plan food production from estimated rather than confirmed demand. Common pain points include:

- **Fixed billing, variable usage:** students who travel, eat out, or skip meals still pay the same flat amount.
- **Low transparency:** students cannot easily see what they are being charged for versus what they actually consumed.
- **Demand guesswork:** the mess prepares food against estimated or historical turnout rather than confirmed participation.
- **Manual reconciliation:** consumption and billing are tracked via registers, WhatsApp, or spreadsheets, forcing rework and disputes.

Why this matters now (contextual relevance): With rising mess costs and limited administrative bandwidth, even a small improvement in consumption visibility can noticeably improve billing fairness and reduce food waste.

4 Proposed Solution

4.1 Overview

Build a web-based “Consumption-to-Bill” platform with the following components:

- **Meal participation & opt-out:** students mark or opt out of upcoming meals before a configured cutoff.
- **Consumption-based billing engine:** eligible meals consumed are converted into transparent, itemized charges.
- **Menu & feedback module:** students view menus and submit ratings/complaints.
- **Admin/ops dashboard:** billing summaries, participation trends, and backlog of complaints.
- **Role-based access control:** student vs. administrator permissions on billing and consumption data.

4.2 Core workflow

1. Student views the published menu and marks participation (or opts out) before the cutoff.
2. System records eligible meal participation and locks it at the cutoff.
3. Meal is served; consumption is logged against the participation record.
4. Billing engine converts eligible consumption into itemized charges at the configured meal rate.
5. Student views updated balance and billing breakdown; admin views demand and reconciliation dashboards.

4.3 Operational constraints for a campus/hostel setting

- Keep the solution usable on low-to-mid range devices via mobile-first, responsive web design, given meal-time usage patterns.
- Avoid dependence on advanced research algorithms; focus on workflow, billing correctness, and automation.
- Keep the billing model configurable per institution, since colleges differ in mess-fee policy.
- Design for deployability using common web hosting and managed databases.

5 Solution Approach

This is an engineering project, so we focus on scalable billing and workflow aspects rather than novel algorithm research.

5.1 Billing/consumption assistance

Eligibility and anomaly checks will be heuristic-based:

- Apply simple, rule-based cutoff and eligibility checks (e.g., no opt-out after the configured window) rather than predictive models.
- Flag unusual participation patterns (e.g., repeated late opt-outs) using basic thresholds, without claiming state-of-the-art detection.
- Show flagged items to staff for review; never auto-charge or auto-waive a disputed record.

5.2 Web architecture

A typical 3-tier web architecture:

- **Frontend:** mobile-responsive UI for students and staff (menu, participation, billing, dashboard).
- **Backend API:** authentication, participation/opt-out, billing calculation, menu and feedback endpoints.
- **Database:** store users, meals, participation records, billing history, and feedback.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features.

6 Evaluation Criterion

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Consumption-to-Billing Latency (CBL): time between a meal window closing and the corresponding charge appearing in the student's billing ledger.

- Target: median CBL ≤ 24 hours during the pilot window.
- Attribution: measured from database timestamps (meal-window close vs. billing ledger update).

6.2 Secondary evaluation metrics

- **Forecast error reduction:** variance between expected and actual meal participation, compared to the pre-SmartMess baseline.
- **Billing dispute rate:** percentage of billing cycles with a student-raised dispute.
- **User clarity:** student-reported clarity of billing using a simple 1–5 feedback question.
- **Reliability:** availability of the platform for login, participation, and billing views during meal hours (e.g., $\geq 99\%$ uptime in pilot).

6.3 Pilot validation plan

- Recruit 10–20 pilot students and 3–5 mess-staff roles in a single hostel.
- Compare metrics before/after within the iteration window by logging baseline estimates via short interviews or existing manual records.

7 Scalability

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in student count and concurrent dashboard usage by:
 - decoupling components (frontend/backend),
 - enabling pagination and indexing for common queries,
 - using stateless API design.
2. **Operationally deployable (practically):** The system should run on common web hosting:
 - managed database,
 - containerized or platform-hosted deployment,
 - CI/CD pipeline for staging/production.

8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework for REST APIs and mobile-responsive pages.
- Managed relational or document database.

- Authentication approach (token-based or session-based).
- Object storage for optional menu images (or local storage for pilot).
- Test framework for unit/integration tests.
- CI runner and staging deployment target.

We will use these mature components to focus on billing correctness, workflow design, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables

9.1 Initial deliverable

- Student authentication + meal participation/opt-out
- Menu publishing and viewing
- Consumption tracking against participation records
- Meal-rate configuration and basic billing calculation
- Student billing view: eligible meals consumed and balance
- CI: build + backend unit tests + linting
- CD to staging with a single command or pipeline job

9.2 Subsequent deliverables

- Admin dashboard for billing, participation, and backlog
- Feedback, ratings, and complaint management
- Food-demand visibility (expected vs. actual participation)
- Improved search/filtering and indexed queries for scale readiness

10 Risks and mitigations

- **Billing-model mismatch:** institution may be unwilling to move off a flat fee. *Mitigation:* keep the billing model fully configurable per institution, with flat-fee mode as a fallback.
- **Unreliable participation signal:** students may forget to mark participation before the cutoff. *Mitigation:* set sensible default cutoffs and consider opt-out-by-default where the institution prefers it.
- **Staff adoption:** mess staff may prefer existing paper/WhatsApp habits. *Mitigation:* replace manual workflows one at a time in Phase 2 rather than all at once.
- **Connectivity in dining areas:** device/network access near hostel mess halls may be inconsistent. *Mitigation:* design for degraded connectivity and confirm minimum device requirements with the pilot site.
- **Billing disputes:** students may forget to opt out and still be charged. *Mitigation:* transparent, itemized billing view and clear cutoff communication.

11 Summary

This project proposes a deployable web platform that makes hostel mess billing fairer and food preparation more efficient by connecting meal consumption to both charges and demand planning. It emphasizes time-to-value through phased, pilot-first delivery, uses CI/CD to support frequent safe releases, and defines success with measurable, attributable evaluation criteria — notably consumption-to-billing latency. It remains focused on engineering workflow, billing correctness, and scalability readiness rather than research-driven novelty.